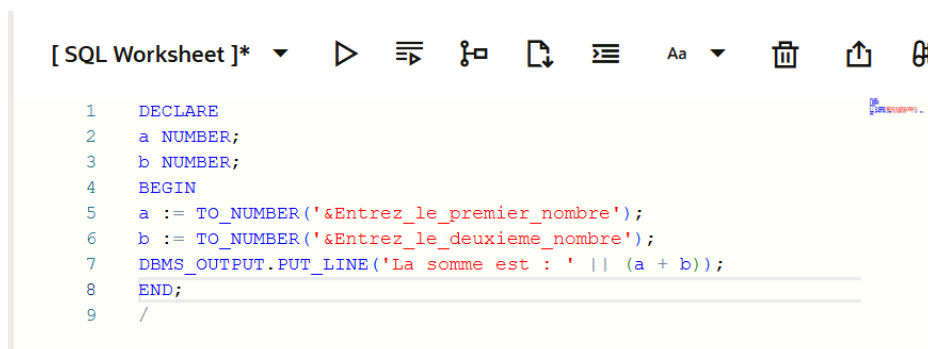


TP4 : Introduction à PL/SQL

Exercice 1 :

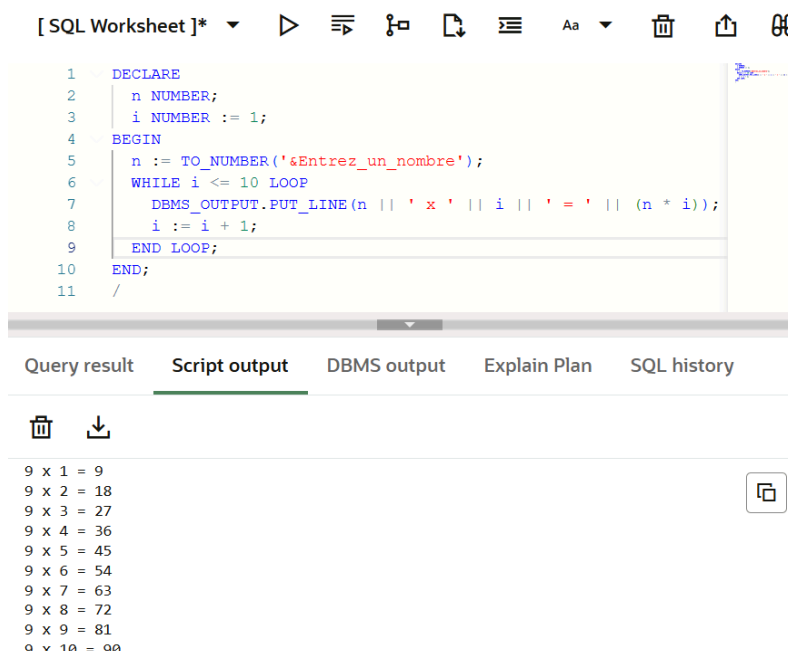
1. Demander à un utilisateur de saisir deux entiers et afficher leur somme.



```
[ SQL Worksheet ]*  ▶  ⏮  🔑  📄  ⏭  Aa  🗑  📤  ⚙

1  DECLARE
2  a NUMBER;
3  b NUMBER;
4  BEGIN
5  a := TO_NUMBER('&Entrez_le_premier_nombre');
6  b := TO_NUMBER('&Entrez_le_deuxieme_nombre');
7  DBMS_OUTPUT.PUT_LINE('La somme est : ' || (a + b));
8  END;
9  /
```

2. Demander à un utilisateur de saisir un nombre et afficher sa table de multiplication.



```
[ SQL Worksheet ]*  ▶  ⏮  🔑  📄  ⏭  Aa  🗑  📤  ⚙

1  DECLARE
2  n NUMBER;
3  i NUMBER := 1;
4  BEGIN
5  n := TO_NUMBER('&Entrez_un_nombre');
6  WHILE i <= 10 LOOP
7  DBMS_OUTPUT.PUT_LINE(n || ' x ' || i || ' = ' || (n * i));
8  i := i + 1;
9  END LOOP;
10 END;
11 /
```

Query result **Script output** DBMS output Explain Plan SQL history

🗑 📄

```
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
```

3. Retourner x^n avec x et n des entiers positifs.

```
[ SQL Worksheet ]*  ▶  ≡  🔑  📄  ≡  Aa  ▼  🗑️  📤  📄
```

```
1  CREATE OR REPLACE FUNCTION puissance(x IN NUMBER, n IN NUMBER)
2  BEGIN
3      IF n = 0 THEN
4          RETURN 1;
5      ELSE
6          RETURN x * puissance(x, n - 1);
7      END IF;
8  END;
9  /
```

4. Procédure anonyme qui calcule la factorielle d'un nombre strictement positif saisi par l'utilisateur avec résultat stocké dans une table resultatFactorielle.

```
[ SQL Worksheet ]*  ▶  ≡  🔑  📄  ≡  Aa  ▼  🗑️  📤  📄
```

```
1  CREATE TABLE resultatFactoriel (nombre NUMBER, factoriel NUMBER);
2  DECLARE
3      n NUMBER;
4      fact NUMBER := 1;
5      i NUMBER;
6  BEGIN
7      n := TO_NUMBER('&Entrez_un_nombre_strictement_positif');
8      IF n <= 0 THEN
9          DBMS_OUTPUT.PUT_LINE('Le nombre doit être strictement positif');
10     ELSE
11         FOR i IN 1..n LOOP
12             fact := fact * i;
13         END LOOP;
14         INSERT INTO resultatFactoriel (nombre, factoriel) VALUES (n, fact);
15         DBMS_OUTPUT.PUT_LINE('La factorielle de ' || n || ' est : ' || fact);
16     END IF;
17 END;
18 /
```

5. Modification du programme précédent pour calculer et stocker dans une tables ResultatsFactorielles les factorielles des 20 premiers nombres entiers.

```
[ SQL Worksheet ]*  ▶  ≡  🔑  📄  ≡  Aa  ▼  🗑️
```

```
1  CREATE TABLE resultatsFactoriels (nombre NUMBER, factoriel NUMBER);
2  DECLARE
3      fact NUMBER := 1;
4  BEGIN
5      FOR i IN 1..20 LOOP
6          fact := 1;
7          FOR j IN 1..i LOOP
8              fact := fact * j;
9          END LOOP;
10         INSERT INTO resultatsFactoriels (nombre, factoriel) VALUES (i, fact);
11     END LOOP;
12     DBMS_OUTPUT.PUT_LINE('Les factorielles ont été calculées et stockées.');
```

Exercice 2 :

Création de la table :

[SQL Worksheet]* ▾ ▶ ⌵ 🔑 ↺ ⌵ Aa ▾ 🗑

```
1 CREATE TABLE emp
2 ( matr NUMBER(10) NOT NULL,
3   nom VARCHAR2(50) NOT NULL,
4   sal number(7,2),
5   adresse VARCHAR2(96),
6   dep NUMBER(10) NOT NULL,
7   CONSTRAINT emp_pk PRIMARY KEY (matr)
8 );
9
```

1. Insertion d'un nouvel employé

[SQL Worksheet]* ▾ ▶ ⌵ 🔑 ↺ ⌵ Aa ▾ 🗑

```
1 DECLARE
2   v_employe emp%ROWTYPE;
3 BEGIN
4   v_employe.matr := 4;
5   v_employe.nom := 'Youcef';
6   v_employe.sal := 2500;
7   v_employe.adresse := 'avenue de la République';
8   v_employe.dep := 92002;
9   INSERT INTO emp VALUES v_employe;
10  DBMS_OUTPUT.PUT_LINE('Employé inséré. ');
11 END;
12 /
```

2. Création de bloc anonyme pour supprimer tous les clients dont le dep est connu en utilisant ROWCOUNT pour afficher le nombre de supprimés.

[SQL Worksheet]* ▾ ▶ ⌵ 🔑 ↺ ⌵ Aa ▾ 🗑

```
1 DECLARE
2   v_nb_lignes NUMBER;
3 BEGIN
4   DELETE FROM emp WHERE dep = 10;
5   v_nb_lignes := SQL%ROWCOUNT;
6   DBMS_OUTPUT.PUT_LINE('Nombre d''employés supprimés : ' || v_nb_lignes);
7 END;
8 /
```

3. Création de bloc anonyme pour afficher la somme des salaires de tous les employés avec curseur explicite.

```
[ SQL Worksheet ]*  ▶ ⌵ 🔑 📄 ⌵ Aa 🗑️ ⏏️ ⌵
```

```
1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_salaire emp.sal%TYPE;
4      v_total emp.sal%TYPE := 0;
5      CURSOR c_salaires IS SELECT sal FROM emp;
6  BEGIN
7      OPEN c_salaires;
8      LOOP
9          FETCH c_salaires INTO v_salaire;
10         EXIT WHEN c_salaires%NOTFOUND;
11         IF v_salaire IS NOT NULL THEN
12             v_total := v_total + v_salaire;
13         END IF;
14     END LOOP;
15     CLOSE c_salaires;
16     DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
17 END;
18 /
```

4. Modification de la procédure précédente pour obtenir le salaire moyen.

```
[ SQL Worksheet ]*  ▶ ⌵ 🔑 📄 ⌵ Aa 🗑️ ⏏️ ⌵
```

```
1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_salaire emp.sal%TYPE;
4      v_total emp.sal%TYPE := 0;
5      v_count NUMBER := 0;
6      CURSOR c_salaires IS SELECT sal FROM emp;
7  BEGIN
8      OPEN c_salaires;
9      LOOP
10         FETCH c_salaires INTO v_salaire;
11         EXIT WHEN c_salaires%NOTFOUND;
12         IF v_salaire IS NOT NULL THEN
13             v_total := v_total + v_salaire;
14             v_count := v_count + 1;
15         END IF;
16     END LOOP;
17     CLOSE c_salaires;
18     IF v_count > 0 THEN
19         DBMS_OUTPUT.PUT_LINE(' Le salaire moyen est de : ' || (v_total / v_count));
20     ELSE
21         DBMS_OUTPUT.PUT_LINE('Aucun salaire de trouvé.');
```

5. Modification des deux procédures précédentes en utilisant cette fois-ci FOR IN.
Somme des salaires :

```
[ SQL Worksheet ]*  ▶ ⌵ 🔑 📄 ⌵ Aa 🗑️ ⏏️ ⌵
```

```
1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_total emp.sal%TYPE := 0;
4      CURSOR c_salaires IS SELECT sal FROM emp;
5  BEGIN
6      FOR ligne IN c_salaires LOOP
7          IF ligne.sal IS NOT NULL THEN
8              v_total := v_total + ligne.sal;
9          END IF;
10     END LOOP;
11     DBMS_OUTPUT.PUT_LINE('La somme des salaires vaut : ' || v_total);
12 END;
13 /
```

Calcul du salaire moyen :

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  📄  ⌵  Aa  ▼  🗑️  🔗  📄

1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_total emp.sal%TYPE := 0;
4      v_count NUMBER := 0;
5      v_moyenne NUMBER;
6      CURSOR c_salaires IS SELECT sal FROM emp;
7  BEGIN
8      FOR ligne IN c_salaires LOOP
9          IF ligne.sal IS NOT NULL THEN
10             v_total := v_total + ligne.sal;
11             v_count := v_count + 1;
12          END IF;
13      END LOOP;
14
15      IF v_count > 0 THEN
16          v_moyenne := v_total / v_count;
17          DBMS_OUTPUT.PUT_LINE('Le salaire moyen est de : ' || v_moyenne);
18      ELSE
19          DBMS_OUTPUT.PUT_LINE('Aucun salaire de trouvé.');
```

6. Création de procédure anonyme permettant d'afficher les noms des employés du département 92000 et 75000 avec un curseur paramétré.

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  📄  ⌵  Aa  ▼  🗑️

1  SET SERVEROUTPUT ON;
2  DECLARE
3      CURSOR c(p_dep emp.dep%TYPE) IS
4          SELECT dep, nom FROM emp WHERE dep = p_dep;
5  BEGIN
6      FOR rec IN c(10) LOOP
7          DBMS_OUTPUT.PUT_LINE('Département: ' || rec.dep || ', Nom employé: ' || rec.nom);
8      END LOOP;
9  END;
10  /
```

Exercice 3 :

Création d'un package PL/SQL pour gérer les clients :

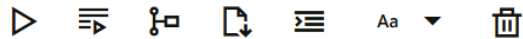
1. Spécification

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  📄  ⌵  Aa  ▼  🗑️

1  CREATE OR REPLACE PACKAGE gestion_clients AS
2      PROCEDURE ajouter_client(p_id_client NUMBER, p_nom VARCHAR2, p_adresse VARCHAR2);
3      PROCEDURE ajouter_client(p_nom VARCHAR2, p_adresse VARCHAR2);
4  END gestion_clients;
5  /
```

2. Corps du package

[SQL Worksheet]*



```
1  CREATE OR REPLACE PACKAGE BODY gestion_clients AS
2      PROCEDURE ajouter_client(p_id_client NUMBER, p_nom VARCHAR2, p_adresse VARCHAR2) IS
3          v_count NUMBER;
4      BEGIN
5          SELECT COUNT(*) INTO v_count FROM clients WHERE id_client = p_id_client;
6          IF v_count > 0 THEN
7              RAISE_APPLICATION_ERROR(-20001, 'ID client déjà utilisé.');
```

```
8          END IF;
9          INSERT INTO clients (id_client, nom, adresse)
10             VALUES (p_id_client, p_nom, p_adresse);
11          DBMS_OUTPUT.PUT_LINE('Ajout du client avec l'ID : ' || p_id_client);
12          COMMIT;
13
14      EXCEPTION
15          WHEN OTHERS THEN
16              DBMS_OUTPUT.PUT_LINE('Erreur ajout client : ' || SQLERRM);
17      END;
18
19      PROCEDURE ajouter_client(p_nom VARCHAR2, p_adresse VARCHAR2) IS
20          v_id_client NUMBER;
21      BEGIN
22          NULL;
23      END;
24  END gestion_clients;
25  /
```